УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «Тестирование программного обеспечения»

Лабораторная работа №1

Вариант 331412

Студенты
Гладкая К. В.
Черкас И. И.
P33141

Преподаватель
Инячина Д. А.
Клименков С. В.

Санкт-Петербург, 2024 г.

Задание лабораторной работы

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Вариант задания

1. Функция sec(x)
2. Программный модуль для работы с кучей (http://www.cs.usfca.edu/~galles/visualization/HeapSort.html)
3. Описание предметной области:

   Зажужжал мотор. Тоненький свист перерос в рев воздуха, вырывающегося в черную пустоту, усеянную невероятно яркими светящимися точками. Форд и Артур вылетели в открытый космос, как конфетти из хлопушки. Глава 8

Код и выполнение задания

1.
```
public class Sec {

    public static double cos(double x, int n) {
        double PI2 = Math.PI * 2;

        if (x >= 0) {
            while (x > PI2) {
                x -= PI2;
            }
        } else if (x < 0) {
            while (x < PI2) {
                x += PI2;
            }
        }

        double result = 1;

        double xx = x * x;
        double pow = xx;
        double fact = 1;

        int sign = -1;

        for (int i = 2; i < n; i += 2){
            fact /= i;
            result += sign * pow * fact;  // (-1)^(n-1) * x^(2n-2) / (2n-2)!
            sign = -sign;
            fact /= (i + 1);
```

```
            pow *= xx;
        }

        return result;

    }
}
```

И тесты

```
public class SecTest {

    @ParameterizedTest(name = "sec({0})")
    @DisplayName("Check PI dots")
    @ValueSource(doubles = {-2 * Math.PI, -Math.PI, 0, Math.PI, 2 * Math.PI})
    void checkPiValues(double param) {
            assertAll(
                    () -> assertEquals(1/Math.cos(param), 1/Sec.cos(param, 100), 0.0001)
            );
    }

    @ParameterizedTest(name = "sec({0})")
    @DisplayName("Check Border dots")
    @ValueSource(doubles = {-2.1 * Math.PI, -1.9 * Math.PI, -1.6 * Math.PI,
            -1.4 * Math.PI, -0.6 * Math.PI, -0.4 * Math.PI, -0.1, 0.1, 0.4 * Math.PI,
            0.6 * Math.PI, 0.9 * Math.PI,  1.1 * Math.PI, 1.4 * Math.PI, 1.6 * Math.PI,
1.9 * Math.PI, 2.1 * Math.PI})
    void checkBorderValues(double param) {
            assertAll(
                () -> assertEquals(1/Math.cos(param), 1/Sec.cos(param, 100), 0.0001)
            );
    }
}
```

Мы использовали разложение cos(x) чтобы найти sec(x) т. к.

$$\frac{1}{\cos x} = \sec x$$

а cos(x) по разложению в степ ряд по Тейлору выглядит так:

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}$$

2.

```
package com.kiwkivyRena.task2;

import lombok.SneakyThrows;

public class HeapSort {

    // Метод сортировки массива с использованием алгоритма Heap Sort
    @SneakyThrows
    public static int[] sort(int[] array) {
        int length = array.length;

        // Строим кучу, начиная с середины массива
        for (int i = length / 2 - 1; i >= 0; i--) {
            heapify(array, length, i);
        }

        // Извлекаем элементы из кучи
        for (int i = length - 1; i > 0; i--) {
            // Перемещаем корень кучи в конец
            int temp = array[0];
            array[0] = array[i];
```

```
            array[i] = temp;

            // Упорядочиваем кучу на уменьшенной куче
            heapify(array, i, 0);
        }

        return array;
    }

    // Процедура упорядочивания поддерева с корнем index в массиве размером size
    private static void heapify(int[] array, int size, int index) {
        int largest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        // Если левый потомок существует и больше текущего корня
        if (left < size && array[left] > array[largest]) {
            largest = left;
        }

        // Если правый потомок существует и больше текущего наибольшего
        if (right < size && array[right] > array[largest]) {
            largest = right;
        }

        // Если наибольший элемент не корень
        if (largest != index) {
            // Обмен корня и наибольшего элемента
            int temp = array[index];
            array[index] = array[largest];
            array[largest] = temp;

            // Рекурсивно упорядочиваем затронутое поддерево
            heapify(array, size, largest);
        }
    }

}
```

Тесты

```
public class HeapSortTest {

    // Проверка сортировки положительных значений
    @Test
    @DisplayName("Check positive values")
    void checkSorting() {
        assertAll(
                () -> assertArrayEquals(new int[]{7, 12, 15, 42}, HeapSort.sort(new
int[]{42, 7, 15, 12})),
                () -> assertArrayEquals(new int[]{2, 4, 6, 8, 10}, HeapSort.sort(new
int[]{8, 2, 4, 10, 6})),
                () -> assertArrayEquals(new int[]{11, 22, 33, 44, 55, 66, 77, 88},
HeapSort.sort(new int[]{44, 55, 11, 22, 77, 88, 66, 33})),
                () -> assertArrayEquals(new int[]{3, 6, 9, 12, 15, 18, 21, 24, 27, 30,
33},
                        HeapSort.sort(new int[]{30, 9, 27, 3, 21, 12, 33, 6, 18, 15,
24}))
        );
    }

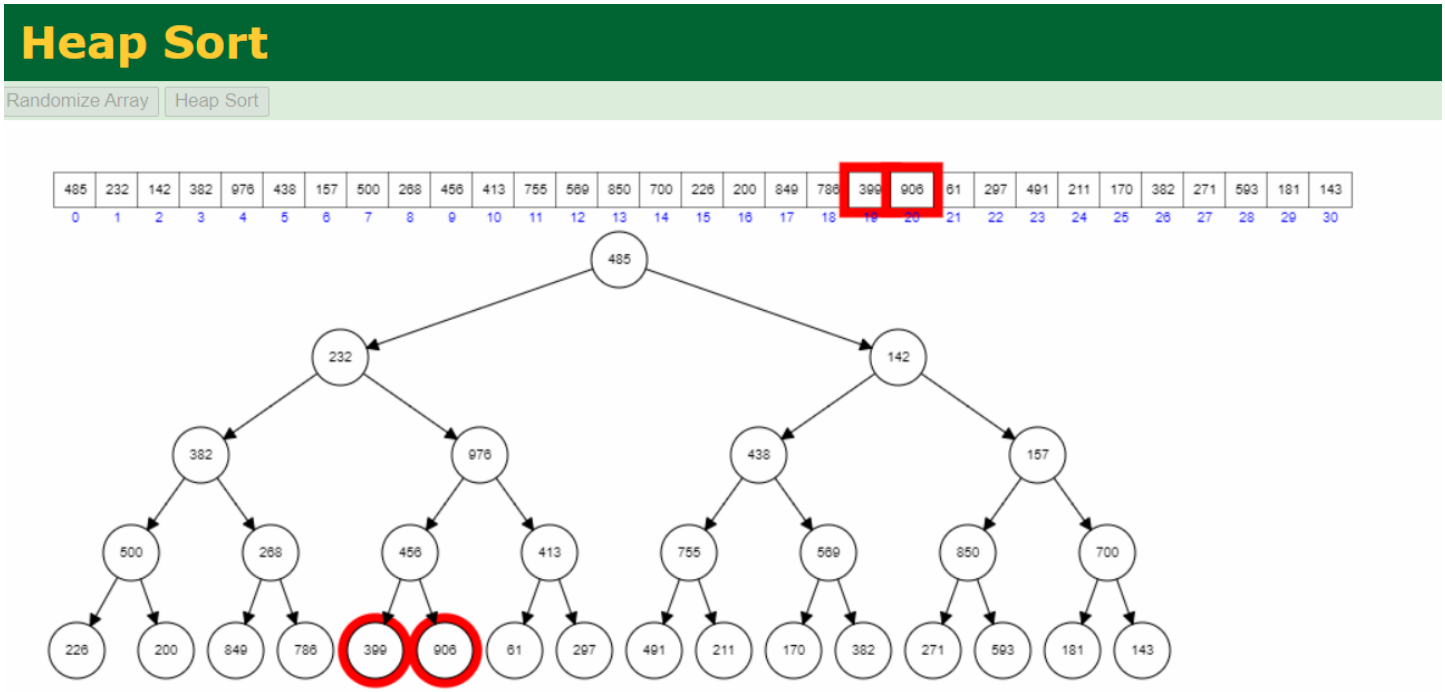
    // Проверка сортировки пустого массива
    @Test
    @DisplayName("Check empty")
    void checkEmpty() {
        assertArrayEquals(new int[]{}, HeapSort.sort(new int[]{}));
    }

    // Проверка сортировки отрицательных значений
    @Test
```

```
    @DisplayName("Check negative values")
    void checkNegativeValues() {
        assertArrayEquals(new int[]{-5, -4, -3, -2, -1}, HeapSort.sort(new int[]{-5, -1,
-4, -3, -2}));
    }

    // Проверка сортировки массива из нулей
    @Test
    @DisplayName("Check zeros")
    void checkZeros() {
        assertArrayEquals(new int[]{0, 0, 0, 0, 0}, HeapSort.sort(new int[]{0, 0, 0, 0,
0}));
    }

    // Проверка обработки сортировки нулевого массива
    @Test
    @DisplayName("Check null")
    void checkNull() {
        assertThrows(NullPointerException.class, () -> HeapSort.sort(null));
    }
}
```



Вывод

Во время выполнения лабораторной работы мы узнали для чего необходимо тестирование, его виды. Вспомнили вычислительную математику. А также рассмотрели различные методы сортировки. И применили параметризированные автоматические тесты на нашу предметную область.